

SQL Interview Questions & Answers

A complete, example-driven reference for Data Analyst & Data Engineering interviews

Prepared for Gaurav · Interview Preparation Guide

Table of Contents

- 1. 1. SQL Basics
- 2. 2. Joins
- 3. 3. Aggregation & Grouping
- 4. 4. Subqueries & CTEs
- 5. 5. Window Functions
- 6. 6. Constraints & Keys
- 7. 7. Normalization
- 8. 8. Indexes & Performance
- 9. 9. Transactions & Concurrency
- 10. 10. Views, Stored Procedures, Functions & Triggers
- 11. 11. Miscellaneous / Tricky Interview Questions

Example schema used throughout this guide:
Customers(CustomerID, Name, City, Country, Email, PhoneNumber) · Orders(OrderID, CustomerID, OrderDate, Amount, Status) · OrderItems(OrderID, ProductID, Quantity) · Products(ProductID, ProductName, ProductCategory, Price, Stock) · Employees(EmployeeID, Name, Salary, ManagerID)

1. SQL Basics

Q1. What is SQL and what are its sub-languages?

SQL (Structured Query Language) is used to create, read, update, and manage data in a relational database. It is divided into sub-languages:

DDL (Data Definition Language) – CREATE, ALTER, DROP, TRUNCATE

DML (Data Manipulation Language) – SELECT, INSERT, UPDATE, DELETE

DCL (Data Control Language) – GRANT, REVOKE

TCL (Transaction Control Language) – COMMIT, ROLLBACK, SAVEPOINT

```
-- DDL
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    Name VARCHAR(100),
    City VARCHAR(50)
);

-- DML
INSERT INTO Customers VALUES (1, 'Rahul Sharma', 'Delhi');
SELECT * FROM Customers;
UPDATE Customers SET City = 'Mumbai' WHERE CustomerID = 1;
DELETE FROM Customers WHERE CustomerID = 1;

-- DCL
GRANT SELECT ON Customers TO analyst_user;
REVOKE SELECT ON Customers FROM analyst_user;

-- TCL
COMMIT;
ROLLBACK;
```

Q2. What is the difference between DROP, DELETE, and TRUNCATE?

DELETE removes rows one at a time, can use WHERE, is logged, and can be rolled back. It does not reset identity/auto-increment counters.

TRUNCATE removes all rows at once, cannot use WHERE, is minimally logged, is faster, and resets identity counters.

DROP removes the entire table structure along with its data.

```
DELETE FROM Orders WHERE Status = 'Cancelled'; -- removes specific rows
TRUNCATE TABLE Orders;                       -- removes all rows, keeps structure
DROP TABLE Orders;                           -- removes table completely
```

Q3. What is the difference between WHERE and HAVING?

WHERE filters individual rows before grouping/aggregation and cannot use aggregate functions. HAVING filters groups after GROUP BY has been applied and can use aggregate functions.

```
SELECT City, COUNT(*) AS TotalCustomers
FROM Customers
WHERE Country = 'India'
GROUP BY City
HAVING COUNT(*) > 5;
```

Q4. What is the difference between CHAR and VARCHAR?

CHAR is a fixed-length data type; it always uses the declared number of characters, padding with spaces if needed. VARCHAR is variable-length; it only uses as much space as the actual data requires, which saves storage for variable-sized text.

```
CREATE TABLE Products (
    ProductCode CHAR(6),          -- always stores exactly 6 characters
    ProductName VARCHAR(100)     -- stores up to 100 characters, uses only what's needed
);
```

Q5. What is the difference between UNION and UNION ALL?

UNION combines results from two queries and removes duplicate rows (it performs an implicit sort/dedupe, so it is slower). UNION ALL combines results and keeps all duplicates, making it faster since no deduplication happens. Both require the same number of columns with compatible data types in each SELECT.

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers;      -- unique cities only

SELECT City FROM Customers
UNION ALL
SELECT City FROM Suppliers;      -- all cities, including duplicates
```

Q6. What is the order of execution of a SQL query?

Even though we write SQL in the order SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, the database actually executes it in this logical order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. This is why you cannot use a column alias defined in SELECT inside a WHERE clause.

```
SELECT CustomerID, COUNT(*) AS OrderCount  -- 5. SELECT
FROM Orders                                -- 1. FROM
WHERE OrderDate >= '2025-01-01'            -- 2. WHERE
GROUP BY CustomerID                        -- 3. GROUP BY
HAVING COUNT(*) > 3                         -- 4. HAVING
ORDER BY OrderCount DESC                   -- 6. ORDER BY
LIMIT 10;                                  -- 7. LIMIT
```

Q7. What is the difference between DISTINCT and GROUP BY?

DISTINCT removes duplicate rows from the result set and is used purely for de-duplication. GROUP BY groups rows sharing the same values so aggregate functions (SUM, COUNT, AVG) can be applied per group. When no aggregation is needed, DISTINCT is simpler and often clearer.

```
SELECT DISTINCT City FROM Customers;

SELECT City, COUNT(*) AS Total
FROM Customers
GROUP BY City;
```

2. Joins

Q8. What are the different types of JOINS in SQL?

INNER JOIN – returns only matching rows from both tables.

LEFT JOIN – returns all rows from the left table plus matches from the right (NULL where no match).

RIGHT JOIN – returns all rows from the right table plus matches from the left.

FULL OUTER JOIN – returns all rows from both tables, matched where possible.

SELF JOIN – a table joined with itself.

CROSS JOIN – returns the Cartesian product of both tables.

```
SELECT o.OrderID, c.Name
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID;

SELECT c.Name, o.OrderID
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID;

SELECT c.Name, o.OrderID
FROM Customers c
FULL OUTER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Q9. How does a SELF JOIN work? Give an example.

A SELF JOIN is used when a table needs to be compared to itself, commonly for hierarchical data such as an employee-manager relationship. The same table is aliased twice.

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

Q10. What is the difference between INNER JOIN and CROSS JOIN?

INNER JOIN combines rows based on a matching condition specified in ON. CROSS JOIN has no join condition and returns every possible combination of rows from both tables (Cartesian product), so the result size is rows(A) * rows(B).

```
SELECT p.ProductName, c.CategoryName
FROM Products p
CROSS JOIN Categories c;    -- every product paired with every category
```

Q11. How do you find rows that exist in one table but not another?

Use a LEFT JOIN and filter for NULL on the right side, or use NOT EXISTS / NOT IN. NOT EXISTS is generally preferred for performance and correct NULL handling.

```
-- Customers who have never placed an order
SELECT c.CustomerID, c.Name
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;

-- Equivalent using NOT EXISTS
SELECT c.CustomerID, c.Name
FROM Customers c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID
);
```

3. Aggregation & Grouping

Q12. What are aggregate functions in SQL? Name a few.

Aggregate functions perform a calculation across a set of rows and return a single value. Common ones: COUNT(), SUM(), AVG(), MIN(), MAX(). They are typically used with GROUP BY.

```
SELECT
    COUNT(*) AS TotalOrders,
    SUM(Amount) AS TotalRevenue,
    AVG(Amount) AS AvgOrderValue,
    MIN(Amount) AS SmallestOrder,
    MAX(Amount) AS LargestOrder
FROM Orders;
```

Q13. How do you count distinct values in a column?

Use COUNT(DISTINCT column_name) to count unique non-NULL values in a column.

```
SELECT COUNT(DISTINCT CustomerID) AS UniqueCustomers
FROM Orders;
```

Q14. How would you find the second highest salary from an Employees table?

Several approaches work. A common one uses a subquery with LIMIT/OFFSET, or DENSE_RANK() for handling ties correctly (so tied top salaries don't skip a rank).

```
-- Approach 1: Subquery
SELECT MAX(Salary) AS SecondHighest
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);

-- Approach 2: LIMIT/OFFSET
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 1 OFFSET 1;

-- Approach 3: DENSE_RANK (handles ties properly)
SELECT Salary FROM (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees
) ranked
WHERE rnk = 2;
```

Q15. How do you find duplicate rows in a table?

Group by the columns that define a duplicate, then filter groups with COUNT greater than 1 using HAVING.

```
SELECT Email, COUNT(*) AS Occurrences
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

Q16. How do you delete duplicate rows while keeping one copy?

Use a window function like ROW_NUMBER() partitioned by the duplicate-defining columns, then delete rows where the row number is greater than 1.

```
WITH ranked AS (
    SELECT CustomerID,
           ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CustomerID) AS rn
    FROM Customers
)
DELETE FROM Customers
WHERE CustomerID IN (SELECT CustomerID FROM ranked WHERE rn > 1);
```

4. Subqueries & CTEs

Q17. What is a subquery? What types exist?

A subquery is a query nested inside another query. It can be:

Scalar – returns a single value.

Row – returns a single row.

Table – returns multiple rows/columns.

Correlated – references columns from the outer query and runs once per outer row.

Non-correlated – runs independently of the outer query.

```
-- Non-correlated subquery
SELECT Name FROM Customers
WHERE CustomerID IN (SELECT CustomerID FROM Orders WHERE Amount > 5000);

-- Correlated subquery
SELECT c.Name
FROM Customers c
WHERE EXISTS (
    SELECT 1 FROM Orders o
    WHERE o.CustomerID = c.CustomerID AND o.Amount > 5000
);
```

Q18. What is a CTE (Common Table Expression) and why use it?

A CTE, defined with WITH, is a named temporary result set that exists only for the duration of the query. It improves readability over nested subqueries and can be referenced multiple times within the same query.

```
WITH HighValueOrders AS (  
    SELECT CustomerID, SUM(Amount) AS TotalSpent  
    FROM Orders  
    GROUP BY CustomerID  
    HAVING SUM(Amount) > 10000  
)  
SELECT c.Name, h.TotalSpent  
FROM HighValueOrders h  
JOIN Customers c ON c.CustomerID = h.CustomerID  
ORDER BY h.TotalSpent DESC;
```

Q19. What is a recursive CTE? Give an example.

A recursive CTE references itself to work through hierarchical or graph-like data, such as an org chart. It has an anchor member (base case) and a recursive member, combined with UNION ALL.

```
WITH RECURSIVE OrgChart AS (  
    -- Anchor: top-level manager (no manager above them)  
    SELECT EmployeeID, Name, ManagerID, 1 AS Level  
    FROM Employees  
    WHERE ManagerID IS NULL  
  
    UNION ALL  
  
    -- Recursive: employees reporting to the previous level  
    SELECT e.EmployeeID, e.Name, e.ManagerID, oc.Level + 1  
    FROM Employees e  
    JOIN OrgChart oc ON e.ManagerID = oc.EmployeeID  
)  
SELECT * FROM OrgChart ORDER BY Level;
```

Q20. What is the difference between EXISTS and IN?

IN compares a value against a list/subquery result and can behave unexpectedly with NULLs in the subquery result. EXISTS checks only whether the subquery returns any rows and stops as soon as one match is found, which is often faster on large correlated checks and handles NULLs safely.

```
-- IN  
SELECT Name FROM Customers  
WHERE CustomerID IN (SELECT CustomerID FROM Orders);  
  
-- EXISTS (generally preferred for correlated checks)  
SELECT Name FROM Customers c  
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID);
```

5. Window Functions

Q21. What are window functions? How are they different from GROUP BY?

Window functions perform calculations across a set of rows related to the current row (a "window") using OVER(), but unlike GROUP BY they do not collapse rows into groups — every original row is still returned, along with the calculated value.

```
SELECT CustomerID, OrderDate, Amount,  
       SUM(Amount) OVER (PARTITION BY CustomerID) AS CustomerTotal  
FROM Orders;  
-- Every order row is kept, with the customer's running total attached.
```

Q22. What is the difference between ROW_NUMBER(), RANK(), and DENSE_RANK()?

ROW_NUMBER() assigns a unique sequential number to each row, even for ties. RANK() assigns the same rank to tied rows but skips subsequent rank numbers. DENSE_RANK() assigns the same rank to tied rows without skipping numbers.

```
SELECT Name, Salary,
       ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNum,
       RANK() OVER (ORDER BY Salary DESC) AS RankNum,
       DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRankNum
FROM Employees;
-- Salaries: 90000, 90000, 85000
-- RowNum:   1, 2, 3
-- RankNum:  1, 1, 3   (skips 2)
-- DenseRankNum: 1, 1, 2   (no skip)
```

Q23. How do LEAD() and LAG() work?

LAG() accesses a value from a previous row, and LEAD() accesses a value from a following row, within the same result set, without needing a self join. Useful for period-over-period comparisons.

```
SELECT OrderDate, Amount,
       LAG(Amount, 1) OVER (ORDER BY OrderDate) AS PreviousOrderAmount,
       LEAD(Amount, 1) OVER (ORDER BY OrderDate) AS NextOrderAmount,
       Amount - LAG(Amount, 1) OVER (ORDER BY OrderDate) AS ChangeFromPrevious
FROM Orders;
```

Q24. How do you find the top N records per group?

Use ROW_NUMBER() (or RANK()) partitioned by the grouping column, then filter in an outer query since window functions can't be used directly in WHERE.

```
WITH ranked AS (
  SELECT CustomerID, OrderID, Amount,
         ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY Amount DESC) AS rn
  FROM Orders
)
SELECT * FROM ranked WHERE rn <= 3;  -- top 3 orders per customer
```

Q25. What is a running total, and how do you calculate one?

A running (cumulative) total adds up values row by row in a specified order. It is calculated with SUM() OVER() using an ORDER BY inside the window, which applies a default frame of "rows from the start up to the current row".

```
SELECT OrderDate, Amount,
       SUM(Amount) OVER (ORDER BY OrderDate
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM Orders;
```

6. Constraints & Keys

Q26. What is the difference between PRIMARY KEY and UNIQUE constraint?

A PRIMARY KEY uniquely identifies each row, does not allow NULLs, and a table can have only one. A UNIQUE constraint also enforces uniqueness but allows one NULL value (in most databases), and a table can have multiple UNIQUE constraints.

```
CREATE TABLE Employees (
  EmployeeID INT PRIMARY KEY,
  Email VARCHAR(100) UNIQUE,
  NationalID VARCHAR(20) UNIQUE
);
```

Q27. What is a FOREIGN KEY and why is it used?

A FOREIGN KEY is a column (or set of columns) in one table that references the PRIMARY KEY of another table, enforcing referential integrity so you cannot insert a value that doesn't exist in the parent table.

```
CREATE TABLE Orders (  
  OrderID INT PRIMARY KEY,  
  CustomerID INT,  
  FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID)  
);
```

Q28. What is a composite key?

A composite key is a primary key made up of two or more columns, used when no single column can uniquely identify a row on its own.

```
CREATE TABLE OrderItems (  
  OrderID INT,  
  ProductID INT,  
  Quantity INT,  
  PRIMARY KEY (OrderID, ProductID)  
);
```

Q29. What is the CHECK constraint used for?

CHECK enforces a condition that every value in a column must satisfy, rejecting any insert or update that violates it.

```
CREATE TABLE Products (  
  ProductID INT PRIMARY KEY,  
  Price DECIMAL(10,2) CHECK (Price > 0),  
  Stock INT CHECK (Stock >= 0)  
);
```

7. Normalization

Q30. What is normalization? Explain 1NF, 2NF, and 3NF briefly.

Normalization is the process of organizing data to reduce redundancy and improve integrity.

1NF – each column holds atomic (indivisible) values, no repeating groups.

2NF – must be in 1NF, and every non-key column depends on the whole primary key (removes partial dependency, relevant for composite keys).

3NF – must be in 2NF, and no non-key column depends on another non-key column (removes transitive dependency).

```
-- Violates 1NF: multiple phone numbers in one field  
-- CustomerID | Phones  
-- 1          | '9876543210, 9123456780'  
  
-- Fixed (1NF): one phone per row  
-- CustomerID | Phone  
-- 1          | 9876543210  
-- 1          | 9123456780
```

Q31. What is denormalization and when would you use it?

Denormalization intentionally introduces redundancy (e.g. combining tables or storing computed values) to reduce the number of joins and improve read performance — commonly used in reporting/analytics or data warehouse tables where read speed matters more than storage efficiency or write-side integrity.

```
-- Normalized: requires a join every time
SELECT o.OrderID, c.Name FROM Orders o JOIN Customers c ON o.CustomerID = c.CustomerID;

-- Denormalized: customer name stored directly on the order for faster reads
CREATE TABLE OrdersDenormalized (
  OrderID INT,
  CustomerID INT,
  CustomerName VARCHAR(100), -- redundant, avoids the join
  Amount DECIMAL(10,2)
);
```

8. Indexes & Performance

Q32. What is an index and why is it useful?

An index is a data structure (usually a B-tree) that speeds up data retrieval by avoiding a full table scan, at the cost of extra storage and slightly slower writes (since the index must also be updated).

```
CREATE INDEX idx_customer_email ON Customers(Email);
CREATE INDEX idx_orders_customer_date ON Orders(CustomerID, OrderDate);
```

Q33. What is the difference between a clustered and non-clustered index?

A clustered index determines the physical storage order of the table's rows — a table can have only one. A non-clustered index is a separate structure with pointers back to the actual rows — a table can have many.

```
-- Clustered index (often the primary key by default)
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY, -- typically becomes the clustered index
  CustomerID INT
);

-- Non-clustered index for faster lookups by CustomerID
CREATE INDEX idx_orders_customer ON Orders(CustomerID);
```

Q34. How do you check how a query is being executed (query plan)?

Use EXPLAIN (or EXPLAIN ANALYZE) before a query to see the execution plan — whether it uses an index scan or a full table scan, join order, and estimated cost — which helps identify performance bottlenecks.

```
EXPLAIN ANALYZE
SELECT * FROM Orders WHERE CustomerID = 101 AND OrderDate > '2025-01-01';
```

Q35. What are some common ways to optimize a slow SQL query?

Key techniques: add indexes on columns used in WHERE/JOIN/ORDER BY; select only needed columns instead of SELECT *; avoid functions on indexed columns in WHERE (which prevents index use); use EXISTS instead of IN for large subqueries; filter early; use appropriate data types; and review the execution plan.

```
-- Avoid: function on indexed column prevents index usage
SELECT * FROM Orders WHERE YEAR(OrderDate) = 2025;

-- Better: sargable condition, index-friendly
SELECT * FROM Orders WHERE OrderDate >= '2025-01-01' AND OrderDate < '2026-01-01';
```

9. Transactions & Concurrency

Q36. What are ACID properties?

Atomicity – a transaction is all-or-nothing.

Consistency – a transaction brings the database from one valid state to another.

Isolation – concurrent transactions don't interfere with each other.

Durability – once committed, changes persist even after a system failure.

```
BEGIN TRANSACTION;

UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;
UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;

COMMIT;  -- both updates succeed together, or neither does
```

Q37. What is the difference between COMMIT, ROLLBACK, and SAVEPOINT?

COMMIT permanently saves all changes made in the current transaction. ROLLBACK undoes changes made since the last COMMIT (or SAVEPOINT). SAVEPOINT marks an intermediate point within a transaction that you can roll back to without undoing the entire transaction.

```
BEGIN TRANSACTION;

UPDATE Inventory SET Stock = Stock - 1 WHERE ProductID = 10;
SAVEPOINT after_stock_update;

UPDATE Orders SET Status = 'Confirmed' WHERE OrderID = 500;
-- something goes wrong here
ROLLBACK TO after_stock_update;  -- undoes only the order status change

COMMIT;
```

Q38. What are transaction isolation levels?

They control how much one transaction can see of another's uncommitted changes:

READ UNCOMMITTED – allows dirty reads (least strict).

READ COMMITTED – prevents dirty reads.

REPEATABLE READ – prevents dirty and non-repeatable reads.

SERIALIZABLE – prevents dirty reads, non-repeatable reads, and phantom reads (most strict, and slowest).

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRANSACTION;
SELECT Balance FROM Accounts WHERE AccountID = 1;
COMMIT;
```

Q39. What is a deadlock?

A deadlock occurs when two or more transactions each hold a lock the other needs, so neither can proceed. Databases typically detect this and automatically roll back one of the transactions to break the cycle.

```
-- Transaction A          -- Transaction B
BEGIN;                    BEGIN;
UPDATE Accounts SET ... WHERE ID=1;  UPDATE Accounts SET ... WHERE ID=2;
UPDATE Accounts SET ... WHERE ID=2;  UPDATE Accounts SET ... WHERE ID=1;
-- A waits for B's lock on ID=2, B waits for A's lock on ID=1: deadlock
```

10. Views, Stored Procedures, Functions & Triggers

Q40. What is a VIEW and why use one?

A view is a saved, virtual table defined by a query. It doesn't store data itself (unless it's a materialized view) but simplifies complex queries, restricts access to specific columns/rows, and provides a consistent interface even if underlying tables change.

```
CREATE VIEW HighValueCustomers AS
SELECT c.CustomerID, c.Name, SUM(o.Amount) AS TotalSpent
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
HAVING SUM(o.Amount) > 50000;

SELECT * FROM HighValueCustomers ORDER BY TotalSpent DESC;
```

Q41. What is the difference between a stored procedure and a function?

A stored procedure can perform actions (INSERT/UPDATE/DELETE), may not return a value (or can return multiple via OUT parameters), and is called with CALL/EXEC. A function must return a single value (or table), is generally used within a SELECT statement, and typically cannot modify data (side-effect free).

```
-- Stored Procedure
CREATE PROCEDURE GetCustomerOrders(IN custId INT)
BEGIN
    SELECT * FROM Orders WHERE CustomerID = custId;
END;
CALL GetCustomerOrders(101);

-- Function
CREATE FUNCTION GetOrderTotal(custId INT) RETURNS DECIMAL(10,2)
BEGIN
    DECLARE total DECIMAL(10,2);
    SELECT SUM(Amount) INTO total FROM Orders WHERE CustomerID = custId;
    RETURN total;
END;
SELECT GetOrderTotal(101);
```

Q42. What is a TRIGGER? Give an example.

A trigger is a block of SQL code that automatically runs in response to an event (INSERT, UPDATE, DELETE) on a table — useful for auditing, enforcing business rules, or keeping related data in sync.

```
CREATE TRIGGER trg_after_order_insert
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
    UPDATE Products
    SET Stock = Stock - NEW.Quantity
    WHERE ProductID = NEW.ProductID;
END;
```

11. Miscellaneous / Tricky Interview Questions

Q43. How do you handle NULL values in SQL?

NULL means "unknown/absent" and is never equal to anything, including another NULL — so use IS NULL / IS NOT NULL for comparisons. Use COALESCE() or ISNULL() to substitute a default value.

```
SELECT Name, COALESCE(PhoneNumber, 'Not Provided') AS Phone
FROM Customers
WHERE PhoneNumber IS NULL;
```

Q44. What is the difference between a temporary table and a CTE?

A temporary table physically stores data for the session/transaction, can have indexes, and can be reused across multiple queries. A CTE exists only for the duration of a single query and cannot be indexed separately.

```
-- Temporary table
CREATE TEMPORARY TABLE TempHighSpenders AS
SELECT CustomerID, SUM(Amount) AS Total FROM Orders GROUP BY CustomerID HAVING SUM(Amount) > 10000;
SELECT * FROM TempHighSpenders;

-- CTE (usable only in the query it's defined in)
WITH TempHighSpenders AS (
    SELECT CustomerID, SUM(Amount) AS Total FROM Orders GROUP BY CustomerID HAVING SUM(Amount) > 10000
)
SELECT * FROM TempHighSpenders;
```

Q45. How do you pivot rows into columns in SQL?

Use conditional aggregation with CASE WHEN inside aggregate functions (portable across most databases), or the native PIVOT operator where supported (e.g. SQL Server).

```
SELECT
    ProductCategory,
    SUM(CASE WHEN MONTH(OrderDate) = 1 THEN Amount ELSE 0 END) AS Jan,
    SUM(CASE WHEN MONTH(OrderDate) = 2 THEN Amount ELSE 0 END) AS Feb,
    SUM(CASE WHEN MONTH(OrderDate) = 3 THEN Amount ELSE 0 END) AS Mar
FROM Orders
GROUP BY ProductCategory;
```

Q46. What is the difference between a correlated subquery and a JOIN?

Both can achieve similar results, but a correlated subquery runs once per outer row and can be slower on large datasets, while a JOIN processes the data set-based and is usually optimized better by the query engine. JOINS are generally preferred when you need columns from both tables in the output.

```
-- Correlated subquery
SELECT c.Name,
    (SELECT SUM(Amount) FROM Orders o WHERE o.CustomerID = c.CustomerID) AS TotalSpent
FROM Customers c;

-- Equivalent JOIN (often more efficient)
SELECT c.Name, SUM(o.Amount) AS TotalSpent
FROM Customers c
LEFT JOIN Orders o ON o.CustomerID = c.CustomerID
GROUP BY c.Name;
```

Q47. How do you find employees who earn more than their manager?

Use a SELF JOIN to compare each employee's salary against their manager's salary in the same table.

```
SELECT e.Name AS Employee, e.Salary, m.Name AS Manager, m.Salary AS ManagerSalary
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmployeeID
WHERE e.Salary > m.Salary;
```

Q48. What is the difference between UNION and JOIN?

JOIN combines columns from two or more tables side by side based on a related column. UNION combines rows (stacks them vertically) from two queries that must have the same number and type of columns.

```
-- JOIN: combines columns
SELECT c.Name, o.Amount FROM Customers c JOIN Orders o ON c.CustomerID = o.CustomerID;

-- UNION: stacks rows
SELECT Name AS Contact FROM Customers
UNION
SELECT Name AS Contact FROM Suppliers;
```

Q49. How do you get the Nth highest value in SQL?

Use OFFSET/LIMIT (or FETCH), or a window function like DENSE_RANK() for a tie-safe approach that works consistently across databases.

```
-- Nth highest salary, e.g. N = 3
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 1 OFFSET 2;    -- OFFSET is N-1

-- Using DENSE_RANK (recommended, handles ties correctly)
SELECT Salary FROM (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees
) t WHERE rnk = 3;
```